MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work No. 7

Operating Systems

Three Small Programs: Processes, Redirection, a Pipe

Supervisors:
Dumitraşcu Marius, University assistant
Reiţman Patricia, University assistant
DISA, FCIM, UTM

Chişinău – 2026

Table of Contents

1 Introduction **3**

2 Tasks and results **3**

2.1 Environment 3

2.2 Part 1 - fork, exec and wait 3

2.3 Part 2 - A zombie, on purpose 5

2.4 Part 3 - Redirection 5

2.5 Part 4 - A pipe 6

2.6 Optional - Three stages 7

3 Conclusions **8**

A Submitted source code **9**

A.1 run.c 9

A.2 zombie.c 10

A.3 redirect.c 10

A.4 pipeline.c 11

A.5 pipeline3.c 12

1 Introduction

Use small Linux C programs to demonstrate process creation, execution, reaping, descriptor redirection and interprocess pipes.

The experiments follow the laboratory handout and were performed inside Ubuntu 26.04.1 LTS in VirtualBox. The figures show actual terminal screenshots taken during execution; complete transcripts and the submitted source files accompany the report.

2 Tasks and results

2.1 Environment

The programs were compiled and executed inside Ubuntu 26.04.1 LTS (ARM64) in VirtualBox as non-root user `misa` on 23 October 2026. The working directory was `~/os-labs-04-08-20261023/lab07`. GCC 15.2.0 compiled all seven programs with C11, `-Wall -Wextra -Werror -O2`. Screenshots and `terminal-output.txt` record the actual runs.

2.2 Part 1 - fork, exec and wait

`run.c` follows the supplied program: the parent prints and flushes its PID, forks, and waits for its child; the child prints its own PID and parent PID and uses execlp to become ls -l. Flushing before fork prevents buffered parent text from being duplicated when output is redirected. The date and nonexistent-program changes are kept as separate sources `run_date.c` and `run_missing.c` so each required case remains runnable.

```
misa@UbuntuVM:lab07$ gcc --version | head -1
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
misa@UbuntuVM:lab07$ make
gcc -std=c11 -D_POSIX_C_SOURCE=200809L -Wall -Wextra -Werror -O2 -o run run.c
gcc -std=c11 -D_POSIX_C_SOURCE=200809L -Wall -Wextra -Werror -O2 -o run_date run_date.c
gcc -std=c11 -D_POSIX_C_SOURCE=200809L -Wall -Wextra -Werror -O2 -o run_missing run_missing.c
gcc -std=c11 -D_POSIX_C_SOURCE=200809L -Wall -Wextra -Werror -O2 -o zombie zombie.c
gcc -std=c11 -D_POSIX_C_SOURCE=200809L -Wall -Wextra -Werror -O2 -o redirect redirect.c
gcc -std=c11 -D_POSIX_C_SOURCE=200809L -Wall -Wextra -Werror -O2 -o pipeline pipeline.c
gcc -std=c11 -D_POSIX_C_SOURCE=200809L -Wall -Wextra -Werror -O2 -o pipeline3 pipeline3.c
misa@UbuntuVM:lab07$ ./run
parent: my PID is 37341
child:  my PID is 37342, my parent is 37341
total 144
-rw-r----- 1 misa misa   253 Oct 23 09:36 Makefile
-rwxrwxr-x 1 misa misa 70848 Oct 23 13:09 pipeline
-rw-r----- 1 misa misa  1294 Oct 23 09:36 pipeline.c
-rwxrwxr-x 1 misa misa 70968 Oct 23 13:09 pipeline3
-rw-r----- 1 misa misa  1475 Oct 23 09:36 pipeline3.c
-rwxrwxr-x 1 misa misa 70944 Oct 23 13:09 redirect
-rw-r----- 1 misa misa   755 Oct 23 09:36 redirect.c
-rwxrwxr-x 1 misa misa 70936 Oct 23 13:09 run
-rw-r----- 1 misa misa   663 Oct 23 09:36 run.c
-rwxrwxr-x 1 misa misa 70944 Oct 23 13:09 run_date
-rw-r----- 1 misa misa   661 Oct 23 09:36 run_date.c
-rwxrwxr-x 1 misa misa 70944 Oct 23 13:09 run_missing
-rw-r----- 1 misa misa   673 Oct 23 09:36 run_missing.c
-rwxrwxr-x 1 misa misa 70984 Oct 23 13:09 zombie
-rw-r----- 1 misa misa   733 Oct 23 09:36 zombie.c
parent: child 37342 ended with status 0
misa@UbuntuVM:lab07$
```

Figure 2.1 - All Linux programs compile in Ubuntu, and run creates a child that executes ls.

```
misa@UbuntuVM:lab07$ ./run_date
parent: my PID is 37605
child:  my PID is 37606, my parent is 37605
Fri Oct 23 13:09:37 UTC 2026
parent: child 37606 ended with status 0
misa@UbuntuVM:lab07$ ./run_missing
parent: my PID is 37607
child:  my PID is 37608, my parent is 37607
exec: No such file or directory
parent: child 37608 ended with status 127
misa@UbuntuVM:lab07$
```

Figure 2.2 - Successful date execution and failed exec of a nonexistent program.

2.2.1 1. Why perror runs only for nosuchprog

Successful execlp replaces the child's process image and does not return to the following line. The same PID now runs ls or date. On failure it returns -1 and sets errno, so `perror("exec")` executes and prints `No such file or directory`. The supplied failure branch then calls `exit(127)`.

2.2.2 2. Whose PID appears as my parent?

In the recorded run, the parent PID was **37341**, the child PID was **37342**, and the child printed **37341** as its parent. getpid returns the caller's own PID; getppid returns its current parent's PID. The parent stays alive and waits during this test.

2.2.3 3. Statuses for ls and nosuchprog

The parent reported child status **0** for ls/date and **127** for the nonexistent program. Linux waitpid returns an encoded wait status, so WEXITSTATUS extracts the normal exit value. The supplied parent itself returns 0 after printing the child's result, including the failure experiment. A general-purpose shell would additionally check normal versus signal termination before interpreting the status.

2.3 Part 2 - A zombie, on purpose

`zombie.c` is the ls version with `sleep(20)` immediately before waitpid. It was started in the background; after two seconds the process table showed the parent sleeping and the exited child unreaped.

```
misa@UbuntuVM:lab07$ ./zombie > /dev/null & zpid=$!
[1] 37875
misa@UbuntuVM:lab07$ sleep 2; zchild=$(pgrep -P "$zpid" | head -1)
misa@UbuntuVM:lab07$ ps -o pid,ppid,stat,cmd -p "$zpid,$zchild"
    PID    PPID STAT CMD
  37875   31888 S    ./zombie
  37876   37875 Z    [ls] <defunct>
misa@UbuntuVM:lab07$
```

Figure 2.3 - The exited ls child is visible with state Z while its parent sleeps.

2.3.1 1. The STAT letter and its meaning

The ls child PID **37876** had state **Z**, and its command appeared as `[ls] <defunct>`. A zombie has finished executing, but the kernel retains its exit information and process-table identity until the parent consumes them. Parent PID **37875** had state S because it was sleeping.

2.3.2 2. What makes the zombie disappear

After the sleep ended, `waitpid(pid, &status, 0)` reaped the child. A later ps query for 37876 printed only the header, with no process row. Waiting for time to pass does not itself remove a zombie; the parent's wait call does.

```
misa@UbuntuVM:lab07$ wait "$zpid"
[1]+  Done                    ./zombie > /dev/null
misa@UbuntuVM:lab07$ ps -o pid,ppid,stat,cmd -p "$zchild"
    PID    PPID STAT CMD
misa@UbuntuVM:lab07$
```

Figure 2.4 - After waitpid, the previous zombie PID no longer has a process-table entry.

2.4 Part 3 - Redirection

The child in `redirect.c` closes descriptor 1, opens out.txt with `O_WRONLY | O_CREAT | O_TRUNC` and mode 0644, then executes ls -l. The parent waits and prints its completion message. The implementation checks fork/open/exec/wait errors.

```
misa@UbuntuVM:lab07$ ./redirect
open gave descriptor 1
done: the listing is in out.txt
misa@UbuntuVM:lab07$ head -3 out.txt
total 144
-rw-r----- 1 misa misa   253 Oct 23 09:36 Makefile
-rw-r--r-- 1 misa misa     0 Oct 23 13:09 out.txt
misa@UbuntuVM:lab07$
```

Figure 2.5 - Descriptor 1 is reused by open and the unchanged ls listing appears in out.txt.

2.4.1 1. Which descriptor open returned, and why

The diagnostic reported **1**. Standard descriptors 0, 1 and 2 were initially open; closing 1 made it the lowest available slot. open chooses that lowest free descriptor. The mode 0644 requested for out.txt already excludes group write, so umask 0002 does not add it back.

2.4.2 2. Why ls no longer prints its listing to the screen

ls still writes to descriptor 1. The child changed what that descriptor refers to before exec; the descriptor remains open across exec and is inherited by the new image. It now points to out.txt. The diagnostic is written to stderr (2), and the parent's own stdout remains unchanged, so both remain visible.

2.4.3 3. Redirecting errors instead

For `2> err.txt`, close descriptor **2** before opening the destination, while leaving 0 and 1 open. A more general implementation can use dup2 to choose descriptor 2 explicitly. Descriptor 1 is stdout and descriptor 2 is stderr.

2.5 Part 4 - A pipe

`pipeline.c` creates a pipe and two children. In the producer, close plus dup connects descriptor 1 to the pipe's write end before exec of ls /etc. In the consumer it connects descriptor 0 to the read end before exec of wc -l. Each child closes the original pipe descriptors after duplication; the parent closes both ends and waits for both children.

```
misa@UbuntuVM:lab07$ ./pipeline
237
misa@UbuntuVM:lab07$ ls /etc | wc -l
237
misa@UbuntuVM:lab07$ ./pipeline3
231
misa@UbuntuVM:lab07$ ls /usr/bin | grep x | wc -l
231
misa@UbuntuVM:lab07$
```

Figure 2.6 - Both the two-stage and three-stage programs match their Ubuntu shell pipelines.

2.5.1 1. Compare with the real shell

Both returned **237**. The important property is equality for the same directory at the same time. ls uses one filename per line when writing into a pipe, and wc -l counts those lines.

2.5.2 2. Which child stays alive after leaving the writer open

Only a temporary copy, `pipeline_broken.c`, omitted the parent's final close of the write end. The submitted pipeline.c remains the correct version. In the process table, broken parent PID **44821** and child **44823**, running `wc -l`, were still sleeping; ls had already exited.

```
misa@UbuntuVM:lab07$ ./pipeline_broken > broken-output.txt & bpid=$!
[1] 44821
misa@UbuntuVM:lab07$ sleep 1; ps -o pid,ppid,stat,cmd -p "$bpid" --ppid "$bpid"
    PID    PPID STAT CMD
  44821   44820 S    ./pipeline_broken
  44823   44821 S    wc -l
misa@UbuntuVM:lab07$ kill -- -"$bpid"; wait "$bpid"
[1]+  Terminated              ./pipeline_broken > broken-output.txt
misa@UbuntuVM:lab07$
```

Figure 2.7 - Leaving the parent writer open keeps wc waiting; the temporary process group is then stopped.

2.5.3 3. What wc is waiting for

wc is waiting for EOF. After buffered bytes have been consumed, a pipe read returns EOF only when every descriptor referring to its write end is closed. The parent's unused open writer prevents EOF while the parent waits for wc. The temporary background process group was terminated after this observation, and no hanging experiment was left running.

2.6 Optional - Three stages

`pipeline3.c` implements `ls /usr/bin | grep x | wc -l` with two pipes and three children. The middle child redirects both stdin and stdout; all children and the parent close all four original pipe ends after the needed duplicates. The parent waits for every child and reports failure if any stage fails.

Both returned **231** in this Ubuntu installation. The three-stage comparison is shown alongside the two-stage result above.

3 Conclusions

The tests connect shell operations to fork, exec and wait. A zombie records an exited child until its parent reaps it. Redirection changes a descriptor in the child so an unchanged program writes to a file. Two- and three-stage pipelines match the shell counts, and the deliberately leaked write descriptor demonstrates that EOF depends on closing every writer.

Bibliography

[1] Operating Systems, FCIM / FAF, Technical University of Moldova. Laboratory work No. 7: *Three Small Programs: Processes, Redirection, a Pipe*. Course handout, 2026–2027.

[2] MIT PDOS. *xv6-riscv*, commit `06aad25c735fd3159bdfae5680be4eab7b1668b2`. Source checked on 23 October 2026. `https://github.com/mit-pdos/xv6-riscv/tree/06aad25c735fd3159bdfae5680be4eab7b1668b2`.

[3] Linux man-pages project. *fork(2), pipe(7), inode(7), path_resolution(7)*. `https://man7.org/linux/man-pages/`.

A Submitted source code

The following listings read the actual submitted source files, so the report and submission use the same implementation. Variant programs are included where relevant.

A.1 run.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command: ls -l */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

A.2 zombie.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command: ls -l */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }

    int status;
    sleep(20); /* Leave the exited child unreaped for observation. */
    waitpid(pid, &status, 0);
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

A.3 redirect.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        close(1);
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); _exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        _exit(127);
    }
    int status;
    if (waitpid(pid, &status, 0) < 0) { perror("waitpid"); return 1; }
    if (!WIFEXITED(status)) return 1;
    if (WEXITSTATUS(status) != 0) return WEXITSTATUS(status);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

A.4 pipeline.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }
    pid_t producer = fork();
    if (producer < 0) { perror("fork"); close(p[0]); close(p[1]); return 1; }
    if (producer == 0) {
        close(1);
        if (dup(p[1]) < 0) { perror("dup"); _exit(1); }
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        _exit(127);
    }
    pid_t consumer = fork();
    if (consumer < 0) {
        perror("fork"); close(p[0]); close(p[1]);
        waitpid(producer, NULL, 0); return 1;
    }
    if (consumer == 0) {
        close(0);
        if (dup(p[0]) < 0) { perror("dup"); _exit(1); }
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        _exit(127);
    }
    close(p[0]); /* The parent uses neither end. */
    close(p[1]); /* Closing the last writer is necessary for EOF. */
    int a, b;
    if (waitpid(producer, &a, 0) < 0) { perror("waitpid"); return 1; }
    if (waitpid(consumer, &b, 0) < 0) { perror("waitpid"); return 1; }
    return !(WIFEXITED(a) && WEXITSTATUS(a) == 0 &&
             WIFEXITED(b) && WEXITSTATUS(b) == 0);
}
```

A.5 pipeline3.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int pipes[2][2];
    pid_t children[3];
    if (pipe(pipes[0]) < 0) { perror("pipe"); return 1; }
    if (pipe(pipes[1]) < 0) {
        perror("pipe"); close(pipes[0][0]); close(pipes[0][1]); return 1;
    }
    char *commands[3][4] = {
        {"ls", "/usr/bin", NULL, NULL},
        {"grep", "x", NULL, NULL},
        {"wc", "-l", NULL, NULL}
    };
    int count = 0, failed = 0;
    for (int stage = 0; stage < 3; stage++) {
        pid_t pid = fork();
        if (pid < 0) { perror("fork"); failed = 1; break; }
        if (pid == 0) {
            if (stage > 0 && dup2(pipes[stage - 1][0], 0) < 0) {
                perror("dup2"); _exit(1);
            }
            if (stage < 2 && dup2(pipes[stage][1], 1) < 0) {
                perror("dup2"); _exit(1);
            }
            for (int i = 0; i < 2; i++) {
                close(pipes[i][0]); close(pipes[i][1]);
            }
            execvp(commands[stage][0], commands[stage]);
            perror("exec"); _exit(127);
        }
        children[count++] = pid;
    }
    for (int i = 0; i < 2; i++) {
        close(pipes[i][0]); close(pipes[i][1]);
    }
    for (int i = 0; i < count; i++) {
        int status;
        if (waitpid(children[i], &status, 0) < 0) { perror("waitpid"); failed = 1; }
        else if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) failed = 1;
    }
    return failed;
}
```